

allocate_unique and allocator_delete

Document: P0316R0
Date: 2017-02-01
Project: Programming Language C++
Audience: Library Evolution Working Group
Author: Miro Knejp (miro.knejp@gmail.com)

Abstract

This paper proposes the addition of the function `allocate_unique()` to complement the functionality of `allocate_shared()` for `unique_ptr`, as well as the addition of the utility `allocator_delete` to hide frequently needed boilerplate when implementing allocator-aware data structures.

Contents

- [Abstract](#)
- [Motivation](#)
- [Proposal](#)
 - [Overview](#)
 - [allocator_delete](#)
 - [allocate_unique](#)
- [Applications Outside of allocate_unique](#)
 - [Node-Based Containers](#)
 - [Type Erasure](#)
- [Open Issues](#)
 - [allocate_unique<T\[\]>](#)
- [Summary](#)
- [Proposed Wording](#)
- [References](#)

Motivation

There is currently an asymmetry in functionality regarding the creation of `shared_ptr` and `unique_ptr`. Both of them can be conveniently created with `make_shared()` and `make_unique()`, respectively, using the default allocator. However if one wishes to use a custom allocator only the former provides a convenient factory in the form of `allocate_shared()` whereas there is no such facility for `unique_ptr` and one has to write quite an amount of non-trivial code to create one's own.

The additions proposed in this paper can be found re-intervened in virtually every library that offers custom allocator support and are already present in standard library implementations for internal use making it obvious this is a frequently sought after functionality that should be readily available in the standard (and *de-facto* already is, albeit not under a well-known name).

The inclusion of `allocator_delete` removes an additional burden for implementing allocator-aware data structures outside the scope of `unique_ptr`. Together with `allocate_unique` it provides the means to conveniently create and destroy objects with custom allocators which are not directly tied to `unique_ptr` lifetimes (such as node-based or type-erasing containers, to name a few).

Proposal

Overview

```
1 template<class T, class Alloc>
2 class allocator_delete {
3 public:
4     using allocator_type = remove_cv_t<Alloc>;
5     using pointer = typename allocator_traits<allocator_type>::pointer;
6
7     template<class OtherAlloc>
```

```

8 allocator_delete(OtherAlloc&& other);
9
10 void operator()(pointer p);
11
12 allocator_type& get_allocator();
13 const allocator_type& get_allocator() const;
14
15 private:
16 allocator_type alloc; // for exposition only
17 };
18
19 template<class T, class Alloc>
20 class allocator_delete<T, Alloc&> {
21 public:
22 using allocator_type = remove_cv_t<Alloc>;
23 using pointer = typename allocator_traits<allocator_type>::pointer;
24
25 allocator_delete(reference_wrapper<Alloc> alloc);
26
27 void operator()(pointer p);
28
29 Alloc& get_allocator() const;
30
31 private:
32 reference_wrapper<Alloc> alloc; // for exposition only
33 };
34
35 template<class T, class OtherAlloc>
36 allocator_delete(OtherAlloc&& alloc)
37 -> allocator_delete<T, typename allocator_traits<OtherAlloc>::template rebind_alloc<T>>;
38
39 template<class T, class Alloc>
40 allocator_delete(reference_wrapper<Alloc> alloc)
41 -> allocator_delete<T, Alloc&>;
42
43 template<class T, class Alloc, class... Args>
44 unique_ptr<T, allocator_delete<T, typename allocator_traits<Alloc>::template rebind_alloc<T>>>
45 allocate_unique(Alloc&& alloc, Args&&... args);
46
47 template<class T, class Alloc, class... Args>
48 unique_ptr<T, allocator_delete<T, Alloc&>>
49 allocate_unique(reference_wrapper<Alloc> alloc, Args&&... args);

```

allocator_delete

The standard library already provides one type intended to be used as the Deleter for `unique_ptr`, namely `default_delete`. The proposed `allocator_delete` is a second standard-provided deleter which does not use the `delete` operator but instead delegates destruction and deletion to a user-provided allocator. By default it stores a copy of the allocator and delegates the necessary operations to the stored copy. A partial specialization for allocator references is provided which only stores a reference to the actual allocator instead of a copy for cases where allocators are stateful and either too big to be carried around in every `unique_ptr` instance or expensive to copy.

`allocator_delete` does not perform rebinding in its call operator. It is an error to instantiate `allocator_delete` with a type `Alloc` not capable of deallocating objects of type `T`. This decision was deliberately made to avoid unnecessary rebinding and copy-constructing of potentially stateful allocators for every single deletion in the call operator.

Because `allocator_delete` must be instantiated only with an allocator type capable of deallocating the intended target type it cannot be naively created from an existing allocator without doing additional work. Class template deduction guides help picking the correct template argument for `Alloc` and hide the required allocator rebinding.

Constructing it with a `reference_wrapper<Alloc>` argument deduces the second template argument to a reference type resulting in `allocator_delete` storing only a reference to an allocator instead of a copy. In this case no rebinding takes place as that would necessitate copying and the user *deliberately* requested reference semantics.

allocate_unique

This is the main motivation of this proposal. The above is required to implement `allocate_unique()` but is useful enough on its own outside the scope of `allocate_unique()` and is therefore proposed as well.

The `allocate_unique()` function is not overly big but tricky enough to implement that a naive approach might be incorrect. Below is an implementation that, to the author's knowledge, is correct and exception safe. Achieving exception safety with the two-

phase creation required with the allocator interface is a common oversight.

```
1 template<class T, class Alloc, class... Args>
2 auto allocate_unique(const Alloc& alloc, Args&&... args) {
3     using traits = typename allocator_traits<Alloc>::template rebind_traits<T>;
4     auto my_alloc = typename traits::allocator_type(alloc);
5     auto hold_deleter = [&my_alloc] (auto p) {
6         traits::deallocate(my_alloc, p, 1);
7     };
8     using hold_t = unique_ptr<T, decltype(hold_deleter)>;
9     auto hold = hold_t(traits::allocate(my_alloc, 1), hold_deleter);
10    traits::construct(my_alloc, hold.get(), forward<Args>(args)...);
11    auto deleter = allocator_delete<T>(my_alloc);
12    return unique_ptr<T, decltype(deleter)>{hold.release(), move(deleter)};
13 }
```

Implementations very similar to the above can be found in numerous libraries and standard implementations. It is a pattern of boilerplate that is repeated frequently enough that it should be included in the standard. Often the intermediary use of a RAI wrapper around the allocate-deallocate pair is forgotten thus resulting in memory leaks if the constructor of T throws. This is a trap people should not have to worry about in the first place.

Applications Outside of allocate_unique

allocator_delete is technically not required to be made available in the standard library's public interface as it can be easily marked as *implementation-defined* in the return type of allocate_unique() as is currently done for the return type of bind(). However its utility shows itself even in other applications for which some examples are given here to convince the reader of their usefulness.

Node-Based Containers

Node-based containers like map or list do typically not store a unique_ptr referencing each and every node. That would store n copies of the deleter which would each have to either copy the allocator for every node or store a reference to the allocator to utilize automatic cleanup. Both are unnecessarily wasteful. The latter establishes a *back reference* from the node to the container, meaning the container becomes expensive to move as all the back references have to be updated. Instead these containers typically manually allocate/deallocate each node, store them as raw pointers, and because the type of a node is virtually never the same type as the payload, rebinding the allocator for the node type is necessary as well. Then for actually allocating each node a procedure similar to the above is performed, followed later by the manual deletion.

This means in practice something like this:

```
1 template<class T, class Alloc>
2 class list {
3     struct node {
4         node* next;
5         T payload;
6         ...
7     }
8     using node_allocator = typename allocator_traits<Alloc>::template rebind_alloc<node>;
9     using traits = allocator_traits<node_allocator>;
10    node_allocator alloc;
11
12    ...
13
14 public:
15     void push_back(T x) {
16         auto hold_deleter = [&alloc] (auto p) {
17             traits::deallocate(alloc, p, 1);
18         };
19         using hold_t = unique_ptr<node, decltype(hold_deleter)>;
20         auto hold = hold_t(traits::allocate(alloc, 1), hold_deleter);
21         traits::construct(alloc, hold.get(), ...);
22         append_node_to_list(hold.release()); // noexcept
23     }
24
25     ~list() {
26         for(auto* node : nodes()) {
27             traits::destroy(alloc, node);
28             traits::deallocate(alloc, node, 1);
29         }
30     }
31 };
```

Compare this to using the utilities proposed in this paper:

```
1 template<class T, class Alloc>
2 class list {
3     struct node {
4         node* next;
5         T payload;
6         ...
7     }
8     using node_allocator = typename allocator_traits<Alloc>::template rebind_alloc<node>;
9     node_allocator alloc;
10
11     ...
12
13 public:
14     void push_back(T x) {
15         auto p = allocate_unique<node>(ref(alloc), ...);
16         append_node_to_list(p.release()); // noexcept
17     }
18
19     ~list() {
20         auto del = allocator_delete<node>(ref(alloc));
21         for(auto* node : nodes()) {
22             del(node);
23         }
24     }
25 };
```

It may not seem like much but the parts that were replaced in the second snippet were the most error-prone. It has much less fiddling around with `allocator_traits` and one did not have to bother themselves with the nature of two-phase initialization and teardown of objects imposed by the allocator interface.

Type Erasure

Containers like `function` or `shared_ptr` employ a technique called *type erasure* where the exact type of the stored object is not visible in the container's type signature. Implementations often rely on using an internal abstract base class from which concrete class templates are derived. If the container has support for user-provided allocators then the allocator has to be stored somewhere as well. But because the type of the allocator is not part of the container's type signature it, too, must be erased. This means both the payload *and* the actual allocator are part of the internal object, often simply combined into a `tuple<Alloc, T>`.

Below is an excerpt showing how such type erasure is frequently implemented:

```
1 class any {
2     struct base {
3         virtual void destroy() noexcept = 0;
4         virtual void do_something() = 0;
5     protected:
6         ~base() = default;
7     }
8
9     template<class Alloc, class T>
10    struct derived : base {
11        derived(const Alloc& alloc, T x);
12        void destroy() noexcept override {
13            using rebind = typename allocator_traits<Alloc>::template rebind_alloc<derived>; // X
14            auto alloc = rebind{move(get<0>(data))}; // X
15            auto* p = this; // X
16            allocator_traits<rebind>::destroy(alloc, p); // X
17            allocator_traits<rebind>::deallocate(alloc, p, 1); // X
18        }
19        void do_something() override { ... }
20        tuple<Alloc, T> data;
21    };
22
23    base* value;
24
25 public:
26    ...
27
28    template<class Alloc, class T>
29    void assign(const Alloc& alloc, T x) {
30        using node = derived<Alloc, T>;
31        using rebind = typename allocator_traits<Alloc>::template rebind_alloc<node>; // X
32        using traits = allocator_traits<rebind>; // X
33        auto node_alloc = rebind{alloc}; // X
```

```

34     auto hold_deleter = [&node_alloc] (auto p) { // X
35         traits::deallocate(node_alloc, p, 1); // X
36     }; // X
37     using hold_t = unique_ptr<node, decltype(hold_deleter)>; // X
38     auto hold = hold_t(traits::allocate(node_alloc, 1), hold_deleter); // X
39     traits::construct(node_alloc, hold.get(), alloc, move(x)); // X
40     if(value) {
41         value->destroy();
42     }
43     value = hold.release();
44 }
45 ~any {
46     if(value) {
47         value->destroy();
48     }
49 }
50 };

```

About half the functional code in this example (marked with x) does the allocator dance. The above can be significantly simplified with the proper tools:

```

1  class any {
2      struct base {
3          virtual void destroy() noexcept = 0;
4          virtual void do_something() = 0;
5      protected:
6          ~base() = default;
7      }
8
9      template<class Alloc, class T>
10     struct derived : base {
11         derived(const Alloc& alloc, T x);
12         void destroy() noexcept override {
13             auto deleter = allocator_delete<derived>(move(get<0>(data))); // X
14             deleter(this); // X
15         }
16         void do_something() override { ... }
17         tuple<Alloc, T> data;
18     };
19
20     base* value;
21
22 public:
23     ...
24
25     template<class Alloc, class T>
26     void assign(const Alloc& alloc, T x) {
27         using node = derived<Alloc, T>;
28         auto p = allocate_unique<node>(alloc, alloc, x); // X
29         if(value) {
30             value->destroy();
31         }
32         value = p.release();
33     }
34     ~any {
35         if(value) {
36             value->destroy();
37         }
38     }
39 };

```

In the altered example only *three lines of code* (marked with x) deal with creation and destruction of the type erased objects with a custom allocator. Note that we pass the allocator twice to `allocate_unique()` as the first argument is the allocator used to allocate the node (automatically rebound for us to the compatible type) and the second argument is forwarded to the allocated node to make a copy available for the `destroy()` method.

Open Issues

`allocate_unique<T[]>`

The current design of `unique_ptr` and the associated deleter means we cannot make `allocator_delete` compatible with the array-based `unique_ptr<T[]>` specialization because there is no way to tell the deleter how many objects to delete. `default_delete` circumvents this problem because the `delete[]` operator knows how many elements were allocated with `new T[]` and it combines

both destruction and deallocation in one operation. In contrast the allocator interface imposes a two-phase cleanup process. Making `allocator_delete` universally compatible with array-based `unique_ptr<T[]>` requires either the addition of a second overload to the deleter's call operator with the signature `void(pointer p, size_t n)` which `unique_ptr<T[]>` would prefer if present, or make it store the number of allocated elements in advance.

Therefore `allocate_unique()` with its first template parameter being of the form `T[]` is currently considered ill-formed until this issue finds a resolution.

Summary

Experience shows that the mechanism abstracted behind `allocate_unique()` is widely re-invented in many projects. Standard library implementations already have it for internal use but people still have to implement their own. As shown in this proposal doing so correctly is tricky and requires more knowledge about the interface of allocators than is usually necessary to actually do the required job. As such the barrier of entry to providing allocator support in a library is often very high as doing it properly involves careful studying of the allocator interface which many consider to be expert-level territory and prefer not to touch with a ten foot pole.

The provided examples show how making internal utilities used to implement `allocate_unique()` available as part of the public interface can greatly help in adding allocator support to other data structures by significantly cutting down on the required boilerplate.

Proposed Wording

These changes are based on [\[N4618\]](#).

1. Change 20.11.1 [unique.ptr] paragraph 6 as follows:

```
template<class T> struct default_delete;
template<class T> struct default_delete<T[]>;

template<class T, class Alloc> class allocator_delete;
template<class T, class Alloc> class allocator_delete<T, Alloc>;

template<class T, class Alloc>
allocator_delete(Alloc&& alloc)
    -> allocator_delete<T, typename allocator_traits<Alloc>::template rebind_alloc<T>>;

template<class T, class Alloc>
allocator_delete(reference_wrapper<Alloc> alloc)
    -> allocator_delete<T, Alloc>;

template<class T, class D = default_delete<T>> class unique_ptr;
template<class T, class D> class unique_ptr<T[], D>;

template<class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
template<class T> unique_ptr<T> make_unique(size_t n);
template<class T, class... Args> unspecified make_unique(Args&&...) = delete;

template<class T, class Alloc, class... Args>
    unique_ptr<T, see below> allocate_unique(Alloc&& alloc, Args&&... args);
template<class T, class Alloc, class... Args>
    unique_ptr<T, see below> allocate_unique(reference_wrapper<Alloc> alloc, Args&&... args);
template<class T, class Alloc, class... Args>
    unspecified allocate_unique(Alloc&& alloc, Args&&... args) = delete;

template<class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;
```

2. Add a new section to 20.11.1 [unique.ptr] as follows:

20.11.1.x Allocator deleter [unique.ptr.allocdltr]

20.11.1.x.1 In general [unique.ptr.allocdltr.general]

The class template `allocator_delete` delegates deletion to client-supplied allocators when used as deleter (destruction policy) for the class template `unique_ptr`.

The template parameter `Alloc` of `allocator_delete` shall satisfy the requirements of `Allocator` (Table 31) unless it is a reference type in which case the requirement applies to the referred-to type.

The template parameter `Alloc` of `allocator_delete` shall not be a rvalue reference type.

The template parameter T of `allocator_delete` may be an incomplete type if the used allocator satisfies the allocator completeness requirements 17.5.3.5.1.

[Note: The intended way of creating `allocator_delete` objects is utilizing the class template deduction guides via `allocator_delete<T>(alloc)` and `allocator_delete<T>(ref(alloc))` (where `alloc` is an allocator) which take care of allocator rebinding. -end note]

20.11.1.x.2 `allocator_delete` [unique.ptr.allocdltr.copy]

```
namespace std {
    template<class T, class Alloc>
    class allocator_delete {
    public:
        using allocator_type = remove_cv_t<Alloc>;
        using pointer = typename allocator_traits<allocator_type>::pointer;

        template<class OtherAlloc>
        allocator_delete(OtherAlloc&& other) noexcept;
        template<class U, class OtherAlloc>
        allocator_delete(const allocator_delete<U, OtherAlloc>& other) noexcept;
        template<class U, class OtherAlloc>
        allocator_delete(allocator_delete<U, OtherAlloc>&& other) noexcept;

        template<class U, class OtherAlloc>
        allocator_delete& operator=(const allocator_delete<U, OtherAlloc>& other) noexcept;
        template<class U, class OtherAlloc>
        allocator_delete& operator=(allocator_delete<U, OtherAlloc>&& other) noexcept;

        void operator()(pointer p);

        Alloc& get_allocator() noexcept;
        const Alloc& get_allocator() const noexcept;

        void swap(allocator_delete& other) noexcept;

    private:
        Alloc alloc; // for exposition only
    };
}
```

The primary class template `allocator_delete` delegates the deletion operation to an instance of `Alloc` stored as part of the deleter.

template<class OtherAlloc> allocator_delete(OtherAlloc&& other) noexcept;

Requires: `OtherAlloc` shall satisfy the requirements of `Allocator` (Table 31).

Effects: Constructs an `allocator_delete` object initializing the stored allocator with `forward<OtherAlloc>(other)`.

Remarks: This constructor shall not participate in overload resolution unless `is_constructible_v<Alloc, OtherAlloc&&>` is true.

template<class U, class OtherAlloc> allocator_delete(const allocator_delete<U, OtherAlloc>& other) noexcept;

Effects: Constructs an `allocator_delete` object initializing the stored allocator with `other.get_allocator()`.

Remarks: This constructor shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `is_constructible_v<Alloc, const remove_reference_t<OtherAlloc>&&>` is true.

template<class U, class OtherAlloc> allocator_delete(allocator_delete<U, OtherAlloc>&& other) noexcept;

Effects: Constructs an `allocator_delete` object initializing the stored allocator with `move(other.get_allocator())`.

Remarks: This constructor shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `is_constructible_v<Alloc, remove_reference_t<OtherAlloc>&&>` is true.

template<class U, class OtherAlloc> allocator_delete& operator=(const allocator_delete<U, OtherAlloc>& other) noexcept;

Effects: `get_allocator() = other.get_allocator()`.

Remarks: This operator shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `is_assignable_v<Alloc, const remove_reference_t<OtherAlloc>&>` is true.

Returns: `*this`.

```
template<class U, class OtherAlloc> allocator_delete& operator=(allocator_delete<U, OtherAlloc>&& other)
noexcept;
```

Effects: `get_allocator() = move(other.get_allocator())`.

Remarks: This constructor shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `is_assignable_v<Alloc, remove_reference_t<OtherAlloc>&&>` is true.

Returns: `*this`.

```
void operator()(pointer p);
```

Effects: Calls `p->~T()` followed by `allocator_traits<Alloc>::deallocate(get_allocator(), p, 1)`.

```
Alloc& get_allocator() noexcept;
const Alloc& get_allocator() const noexcept;
```

Returns: A reference to the stored allocator.

```
void swap(allocator_delete& other) noexcept;
```

Requires: `get_allocator()` shall be swappable (17.5.3.2) and shall not throw an exception under `swap`.

Effects: Invokes `swap` on the stored allocators of `*this` and `other`.

20.11.1.x.3 `allocator_delete<T, Alloc>` [unique.ptr.allocdltr.ref]

```
namespace std {
    template<class T, class Alloc>
    class allocator_delete<T, Alloc> {
    public:
        using allocator_type = remove_cv_t<Alloc>;
        using pointer = typename allocator_traits<allocator_type>::pointer;

        template<class OtherAlloc>
        allocator_delete(reference_wrapper<OtherAlloc> other) noexcept;
        template<class U, class OtherAlloc>
        allocator_delete(allocator_delete<U, OtherAlloc>& other) noexcept;
        template<class U, class OtherAlloc>
        allocator_delete& operator=(allocator_delete<U, OtherAlloc>& other) noexcept;

        void operator()(pointer p);

        Alloc& get_allocator() const noexcept;

        void swap(allocator_delete& other) noexcept;

    private:
        reference_wrapper<Alloc> alloc; // for exposition only
    };
}
```

A specialization for allocator lvalue references is provided to delegate deletion to a referred-to allocator instead of to a stored copy.

```
template<class OtherAlloc> allocator_delete(reference_wrapper<OtherAlloc> other) noexcept;
```

Requires: `OtherAlloc` shall satisfy the requirements of `Allocator` (Table 31).

Effects: Constructs an `allocator_delete` object storing a reference to `other.get()`.

Remarks: This constructor shall not participate in overload resolution unless `OtherAlloc&` is implicitly convertible to `Alloc&`.

```
template<class U, class OtherAlloc> allocator_delete(allocator_delete<U, OtherAlloc>& other) noexcept;
```

Effects: Constructs an `allocator_delete` object storing a reference to `other.get_allocator()`.

Remarks: This constructor shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `OtherAlloc&` is implicitly convertible to `Alloc&`.

```
template<class U, class OtherAlloc> allocator_delete& operator=(allocator_delete<U, OtherAlloc&> other)
noexcept;
```

Effects: Rebinds the stored allocator reference to `other.get_allocator()`.

Remarks: This operator shall not participate in overload resolution unless:

- `U*` is implicitly convertible to `T*`, and
- `OtherAlloc&` is implicitly convertible to `Alloc&`.

Returns: `*this`.

```
void operator()(pointer p);
```

Effects: Calls `p->~T()` followed by `allocator_traits<Alloc>::deallocate(get_allocator(), p, 1)`.

```
Alloc& get_allocator() const noexcept;
```

Returns: `alloc.get()`.

```
void swap(allocator_delete& other) noexcept;
```

Effects: Swaps the allocator references of `*this` and `other`.

3. Append new paragraphs to section 20.11.1.4 [unique.ptr.create] as follows:

```
template<class T, class Alloc, class... Args> unique_ptr<T, see below > allocate_unique(Alloc&& alloc,
Args&&... args);
```

Requires: The expression `::new (pv) T(forward<Args>(args)...) where pv has type void* and points to storage suitable for holding an object of type T, shall be well formed. Alloc shall satisfy the requirements of Allocator (17.5.3.5).`

Effects: Allocates memory suitable for holding an object of type `T` using a copy of `alloc` and constructs an object in that memory via the placement *new-expression* `::new (pv) T(forward<Args>(args)...) .`

Returns: An instance of `unique_ptr<T, allocator_delete<T, A>>` with ownership of the allocated object and the deleter holding the allocator used for allocation, where `A` has type `allocator_traits<Alloc>::rebind_alloc<T>`.

Postcondition: `get() != nullptr`.

Throws: Any exception thrown from `Alloc::allocate` or the constructor of `T`.

Remarks: This function shall not participate in overload resolution unless `T` is not an array.

```
template<class T, class Alloc, class... Args> unique_ptr<T, see below >
allocate_unique(reference_wrapper<Alloc> alloc, Args&&... args);
```

Requires: The expression `::new (pv) T(forward<Args>(args)...) , where pv has type void* and points to storage suitable for holding an object of type T, shall be well formed. Alloc shall satisfy the requirements of Allocator (17.5.3.5). Alloc shall be capable of allocating memory suitable for holding an object of type T.`

Effects: Allocates memory suitable for holding an object of type `T` using `alloc.get()` directly and constructs an object in that memory via the placement *new-expression* `::new (pv) T(forward<Args>(args)...) .`

Returns: An instance of `unique_ptr<T, allocator_delete<T, Alloc&>>` with ownership of the allocated object and the deleter initialized with `ref(alloc)`.

Postcondition: `get() != nullptr`.

Throws: Any exception thrown from `Alloc::allocate` or the constructor of `T`.

Remarks: This function shall not participate in overload resolution unless `T` is not an array.

```
template<class T, class Alloc, class... Args> unspecified allocate_unique(Alloc&& alloc, Args&&... args) =  
delete;
```

Remarks: This function shall not participate in overload resolution unless τ is an array.

References

[N4618] [Working Draft, Standard for Programming Language C++](#)